# Using CPLEX C API

Throughout the course every program is compiled with **CMake**, using the same `CMakeLists.txt` on Linux, macOS and Windows.

## The `CMakeLists.txt` file

Every example in `Examples/Basic_Models_C` comes with a `CMakeLists.txt`. To create a new project:

1. Put your code in a single `.cpp` file (plus as many `.h`/`.hpp` files as you want) in a folder
2. Copy the `CMakeLists.txt` of any example into the same folder
3. Open it and set `NAME_DEFAULT` to the name of your `.cpp` file, without the extension

```
# for main.cpp
set(NAME_DEFAULT "main")
# for transport_timing.cpp
set(NAME_DEFAULT "transport_timing")
```

   The executable will have the same name (`main`, `main.exe`, ...)

You do not need to edit anything else: the file finds the CPLEX installation by itself, looking in this order at

1. the `CPLEX_DIR` CMake variable (e.g. `-DCPLEX_DIR=/path/to/CPLEX_Studio<version>`)
2. the `CPLEX_DIR` environment variable
3. the `CPLEX_STUDIO_DIR<version>` environment variables (created by the CPLEX installer on Windows)
4. the default installation paths (if several versions are installed, the most recent is used)

```
# Linux
/opt/ibm/ILOG/CPLEX_Studio<version>/
# MacOS
/Applications/CPLEX_Studio<version>/
# Windows
C:\Program Files\IBM\ILOG\CPLEX_Studio<version>\
```

If CPLEX is installed elsewhere, use option 1 or 2.

In your source code, include CPLEX with

```
#include <ilcplex/cplex.h>
```

## Linux / MacOS

1. Make sure you already installed CPLEX
2. Install a C++ compiler and CMake (version 3.18 or newer)

```
# Ubuntu / Debian
sudo apt install build-essential cmake
# MacOS (requires Homebrew, https://brew.sh)
xcode-select --install
brew install cmake
```

3. Write your program and the `CMakeLists.txt` as explained above
4. From within the folder where the code is, configure (only the first time) and compile

```
cmake -S . -B build      # configure: creates the "build" folder
cmake --build build      # compile: run this again after every change
```

The configure step prints the CPLEX installation it found, e.g.

```
-- CPLEX: /Applications/CPLEX_Studio2212
-- Compiling: main.cpp
```

5. Execute the program

```
# HERE USE THE NAME YOU SET IN NAME_DEFAULT
./build/main
```

6. To start over (e.g. after installing a new CPLEX version), delete the `build` folder

**Notes**

- You can also choose the file to compile from the command line, without editing `CMakeLists.txt`: `cmake -S . -B build -DNAME=other_file`. This choice is remembered in `build`: to go back to `NAME_DEFAULT` delete the `build` folder.
- Apple Silicon Macs with only the Intel version of CPLEX: the executable is compiled for Intel and runs through Rosetta (CMake prints a warning, it is fine).

**Using VSCode (Linux/MacOS)**

1. Install the extensions **C/C++** and **CMake Tools** (both by Microsoft)
2. Open the folder containing `CMakeLists.txt` (`File > Open Folder...`)
3. When asked, select a compiler ("kit"): GCC on Linux, Clang on macOS
4. Build with `F7` (or the **Build** button in the status bar), run with `Shift+F5`

CMake Tools passes the CPLEX include path to IntelliSense automatically, so `#include <ilcplex/cplex.h>` is found without any `c_cpp_properties.json`. If it is still underlined in red, open the command palette (`Ctrl/Cmd+Shift+P`) and run `C/C++: Change Configuration Provider...` → `CMake Tools`.

## Windows (Visual Studio)

On Windows CPLEX only works with the Microsoft compiler (MSVC, 64 bit). MinGW/GCC is **not** supported. We use **Visual Studio**, not VSCode/Visual Studio Code.

**Installation (only once)**

1. Install CPLEX through the IBM SkillsBuild initiative, keeping the default installation folder (`C:\Program Files\IBM\ILOG\CPLEX_Studio<version>`). The installer also creates the `CPLEX_STUDIO_DIR<version>` environment variable and adds CPLEX to the `PATH`.
2. Download Visual Studio **Community** (free) from here and run the installer
3. In the **Workloads** tab select **Desktop development with C++** (it. "Sviluppo di applicazioni desktop con C++"). In the panel on the right, check that these components are selected (they are by default):
   - **MSVC … C++ x64/x86 build tools** (latest version)
   - **C++ CMake tools for Windows**
   - a **Windows 11 SDK** (or Windows 10 SDK)

   These three are the only components we need. For a smaller installation, select only them in the **Individual components** tab instead of the whole workload. CMake (and Ninja) come with Visual Studio: nothing else needs to be installed.

4. **Restart the computer** (or at least log out and back in), so that Visual Studio sees the environment variables created by the CPLEX installer

**Compiling and running**

1. Write your program and the `CMakeLists.txt` as explained above (or use one of the examples)
2. Start Visual Studio and choose **Open a local folder** (or `File > Open > Folder...`), then select the folder containing `CMakeLists.txt`. Do not use "Create a new project".
3. Visual Studio detects `CMakeLists.txt` and configures the project automatically. Wait until the **Output** window (`View > Output`, "Show output from: CMake") shows

   ```
   1> -- CPLEX: C:/Program Files/IBM/ILOG/CPLEX_Studio2212
   1> -- Compiling: main.cpp
   1> CMake generation finished.
   ```

4. In the toolbar check that the configuration is **x64-Debug** (the default; any `x64-...` configuration works, `x86` ones do not)
5. In the **Select Startup Item** drop-down next to the green ▶ button, choose your executable (e.g. `main.exe`)
6. Run:
   - `Ctrl+F5` (**Start Without Debugging**): compiles if needed, runs, and keeps the console window open at the end
   - `F5` (**Start Debugging**): same, with the debugger (breakpoints, step-by-step, ...)
   - `Ctrl+Shift+B` (**Build** > **Build All**): compiles only
7. To compile a different file, change `NAME_DEFAULT` in `CMakeLists.txt` and save: Visual Studio reconfigures by itself.

**Notes**

- The program runs inside the build folder, `out\build\x64-Debug`: files written by the program (e.g. the `.lp` model) end up there, and files it reads must be there too. To run it from the source folder instead, right-click `CMakeLists.txt` → **Add Debug Configuration** (or `Debug > Debug and Launch Settings for <name>`) and add to the configuration in `launch.vs.json`:

  ```
  "currentDir": "${workspaceRoot}"
  ```

- The CPLEX DLL (`cplex<version>.dll`) is copied next to the executable at every build, so the program also starts outside Visual Studio (double click on `out\build\x64-Debug\main.exe`).
- To start over, use `Project > Delete Cache and Reconfigure` (or close Visual Studio and delete the `out` and `.vs` folders).

**Troubleshooting (Windows)**

- **`CPLEX not found`**: Visual Studio was started before installing CPLEX (restart the computer), or CPLEX is not in the default folder. In the latter case go to `Project > CMake Settings` (it opens `CMakeSettings.json`), and in **CMake command arguments** write

  ```
  -DCPLEX_DIR="D:/my/path/CPLEX_Studio2212"
  ```

  (use `/` instead of \). Alternatively create a `CPLEX_DIR` environment variable with that path (`Settings > System > About > Advanced system settings > Environment Variables`) and restart Visual Studio.
- **`On Windows CPLEX requires the 64-bit MSVC compiler`**: an `x86` configuration is selected; choose `x64-Debug`.

- **`cplex<version>.dll was not found`** when running the program: rebuild (`Ctrl+Shift+B`), or add `C:\Program Files\IBM\ILOG\CPLEX_Studio<version>\cplex\bin\x64_win64` to the `PATH` environment variable.
- **`#include <ilcplex/cplex.h>` underlined in red**: IntelliSense takes the include path from CMake. Wait for the configuration to finish; if the Output window shows a CMake error, fix that first.
- **No executable in Select Startup Item**: the CMake configuration failed; read the error in the Output window.

**Command line (optional)**

From the **Developer PowerShell for VS** (Start menu) you can use the same commands as Linux/macOS:

```
cmake -S . -B build
cmake --build build --config Release
.\build\Release\main.exe
```